

1. Complexity Analysis

Priority Topic

- A Big-O, Big-Theta, Big-Omega
- A Time complexity analysis
- A Space complexity analysis
- A Worst / average / best case
- A Amortized analysis
- A Recursion complexity
- A Master theorem basics
- B Recurrence solving
- B Logarithmic complexity intuition
- C Potential method for amortized analysis

You must be able to quickly analyze code like:

```
for (int i = 1; i <= n; i *= 2) {}
```

```
for (int i = 0; i < n; i++)  
    for (int j = i; j < n; j++) {}
```

2. Arrays

Priority Topic

- A Prefix sum
- A Suffix sum
- A Difference array
- A Kadane's algorithm
- A Two pointers
- A Sliding window

Priority Topic

- A Sorting + greedy on arrays
- A Frequency arrays
- A Hashing with arrays
- A In-place modification
- A Rotate array
- A Merge intervals
- A Subarray sum problems
- A Maximum/minimum subarray
- A Product except self
- A Partitioning
- B Prefix minimum/maximum
- B Contribution technique
- B Coordinate compression
- B Sweep line basics
- B Mo's algorithm
- C Offline queries on arrays
- C Divide and conquer optimization on arrays

Must-practice patterns:

Subarray sum = k

Longest subarray with condition

Maximum subarray sum

Minimum window

Merge intervals

Sort by custom condition

Prefix/suffix contribution

3. Strings

Priority Topic

A	Character frequency
A	Two pointers on strings
A	Sliding window on strings
A	Palindrome checking
A	String reversal
A	Substring problems
A	Anagram problems
A	Longest common prefix
A	Basic hashing
B	KMP algorithm
B	Z algorithm
B	Rabin-Karp
B	Rolling hash
B	Trie
B	Manacher's algorithm
C	Suffix array
C	Suffix automaton
C	Aho-Corasick
C	Palindromic tree / Eertree

For placements, must know:

Sliding window

Palindrome

Anagram

KMP basics

Trie

String hashing basics

For CP/high-level companies, add:

Z algorithm

Manacher

Suffix array

Aho-Corasick

4. Linked List

Priority Topic

- A Singly linked list
- A Doubly linked list
- A Reverse linked list
- A Fast and slow pointers
- A Detect cycle
- A Find middle
- A Merge two sorted lists
- A Remove nth node from end
- A Intersection of linked lists
- A Palindrome linked list
- B Reverse nodes in k-group
- B Copy list with random pointer
- B Flatten linked list
- B LRU cache using DLL + hashmap

Must-practice:

Reverse list

Cycle detection

Middle node

Merge lists

LRU cache

Copy random pointer

5. Stack

Priority Topic

- | | |
|---|------------------------------------|
| A | Basic stack operations |
| A | Balanced parentheses |
| A | Next greater element |
| A | Previous greater element |
| A | Next smaller element |
| A | Previous smaller element |
| A | Monotonic stack |
| A | Min stack |
| A | Evaluate postfix/prefix expression |
| B | Largest rectangle in histogram |
| B | Maximal rectangle |
| B | Stock span |
| B | Asteroid collision |
| B | Decode string |
| B | Remove duplicate letters |

Important pattern:

Whenever you need nearest greater/smaller on left/right, think monotonic stack.

6. Queue and Deque

Priority Topic

- A Queue basics
- A Circular queue
- A Deque
- A BFS using queue
- A Sliding window maximum
- B Monotonic deque
- B First negative in every window
- B Rotten oranges
- B Task scheduler
- C 0-1 BFS using deque

Must-practice:

Sliding window maximum

BFS level order

Rotten oranges

0-1 BFS

7. Hashing

Priority Topic

- A unordered_map
- A unordered_set
- A Frequency counting

Priority Topic

- A Pair sum / two sum
- A Subarray sum with hashmap
- A Longest consecutive sequence
- A Detect duplicates
- B Custom hash
- B Hashing pairs
- B Coordinate compression
- B Rolling hash
- C Double hashing
- C Universal hashing basics

Must-practice:

Two sum

Subarray sum equals k

Longest consecutive sequence

Group anagrams

Frequency-based sorting

8. Binary Search

Priority Topic

- A Basic binary search
- A Lower bound
- A Upper bound
- A First occurrence
- A Last occurrence

Priority Topic

- A Search in rotated sorted array
- A Binary search on answer
- A Minimum possible maximum
- A Maximum possible minimum
- B Floating-point binary search
- B Ternary search
- B Binary search with greedy check
- B Binary lifting basics
- C Parallel binary search
- C Parametric search advanced

Most important placement pattern:

Binary search on answer + feasibility check

Practice problems:

Book allocation

Aggressive cows

Koko eating bananas

Minimum days to make bouquets

Split array largest sum

Median of two sorted arrays

9. Sorting

Priority Topic

- A Selection sort
- A Bubble sort
- A Insertion sort

Priority Topic

- A Merge sort
- A Quick sort
- A Counting sort
- A Custom comparator
- A Sorting intervals
- A Stable vs unstable sort
- B Radix sort
- B Bucket sort
- B Topological sorting
- B Sort + greedy
- C External sorting
- C TimSort idea

You should know implementation and complexity of:

Merge sort

Quick sort

Counting sort

std::sort internals idea

10. Recursion and Backtracking**Priority Topic**

- A Basic recursion
- A Recursion tree
- A Backtracking template
- A Subsets

Priority Topic

- A Permutations
- A Combinations
- A N-Queens
- A Sudoku solver
- A Rat in a maze
- A Generate parentheses
- B Word search
- B Palindrome partitioning
- B Combination sum
- B Hamiltonian path basics
- C Meet-in-the-middle
- C Branch and bound

Basic template:

```
void backtrack(state) {  
    if (base_condition) {  
        store_answer();  
        return;  
    }  
  
    for (choice : choices) {  
        make_choice();  
        backtrack(new_state);  
        undo_choice();  
    }  
}
```

11. Bit Manipulation

Priority Topic

- A AND, OR, XOR
- A Left shift, right shift
- A Check if bit is set
- A Set bit
- A Clear bit
- A Toggle bit
- A Count set bits
- A Power of two check
- A XOR properties
- A Single number problems
- B Bitmasking
- B Subsets using bitmask
- B DP with bitmask
- B Brian Kernighan algorithm
- B Lowest set bit
- B XOR basis basics
- C SOS DP
- C Fast Walsh-Hadamard Transform
- C Bitset optimization

Must-practice:

Single number

Missing number

Subsets

XOR from 1 to n

Count bits

Bitmask DP TSP

12. Mathematics for DSA

Priority Topic

A GCD

A LCM

A Euclidean algorithm

A Prime checking

A Sieve of Eratosthenes

A Modular arithmetic

A Modular exponentiation

A Factorial modulo

A nCr basics

A Divisors

A Prime factorization

B Modular inverse

B Fermat's little theorem

B Extended Euclidean algorithm

B Chinese Remainder Theorem

B Inclusion-exclusion

B Pigeonhole principle

B Catalan numbers

Priority Topic

- B Matrix exponentiation
- C Mobius function
- C Euler totient function
- C Number theoretic transform
- C Miller-Rabin primality
- C Pollard Rho
- C Burnside lemma

For placements, must know:

GCD

LCM

Sieve

Modulo

Fast power

nCr

Prime factorization

For CP, add:

CRT

Euler phi

Mobius

Matrix exponentiation

Miller-Rabin

13. Trees

Priority Topic

- A Binary tree basics

Priority Topic

A	Tree traversal
A	Preorder
A	Inorder
A	Postorder
A	Level order
A	Height/depth
A	Diameter of tree
A	Balanced tree check
A	Lowest common ancestor
A	Root-to-leaf paths
A	Path sum
A	Binary search tree
A	Insert/search/delete in BST
A	Validate BST
B	Serialize/deserialize tree
B	Construct tree from traversals
B	Vertical order traversal
B	Boundary traversal
B	Morris traversal
B	Binary lifting for LCA
B	Tree DP
C	Heavy-Light Decomposition
C	Centroid decomposition

Priority Topic

C Euler tour + segment tree

C Link-cut tree

Must-practice:

Tree traversals

Diameter

LCA

Path sum

Validate BST

Build tree

Serialize/deserialize

Tree DP basics

14. Binary Search Tree and Balanced Trees

Priority Topic

A BST property

A Search in BST

A Insert in BST

A Delete in BST

A Inorder gives sorted order

A Floor and ceil

A Kth smallest

B AVL tree idea

B Red-black tree idea

B Treap

B Ordered set

Priority Topic

C Splay tree

C Persistent BST

For C++ placements, understand:

set/map are usually implemented using red-black trees.

15. Heap / Priority Queue

Priority Topic

A Min heap

A Max heap

A Heapify

A Priority queue

A K largest/smallest elements

A Merge k sorted lists

A Running median

A Top K frequent elements

B Custom comparator

B Heap sort

B Two heaps technique

B Dijkstra using priority queue

C Fibonacci heap idea

C Pairing heap idea

Must-practice:

Kth largest

Top K frequent

Merge k lists

Median from data stream

Task scheduler

Dijkstra

16. Graphs

This is one of the most important sections.

Graph Basics

Priority Topic

- A Directed graph
- A Undirected graph
- A Weighted graph
- A Unweighted graph
- A Adjacency matrix
- A Adjacency list
- A Degree
- A Connected components
- A Graph traversal

BFS / DFS

Priority Topic

- A BFS
- A DFS
- A Connected components
- A Cycle detection in undirected graph
- A Cycle detection in directed graph
- A Bipartite graph

Priority Topic

- A Flood fill
- A Matrix BFS/DFS
- B Multi-source BFS
- B 0-1 BFS
- B DFS timestamps
- B Bridges
- B Articulation points

Shortest Path

Priority Topic

- A BFS shortest path in unweighted graph
- A Dijkstra
- B Bellman-Ford
- B Floyd-Warshall
- B 0-1 BFS
- B Shortest path in DAG
- C Johnson's algorithm
- C A* search

Topological Sort

Priority Topic

- A Topological sorting using DFS
- A Kahn's algorithm
- A Course schedule problems
- B Detect cycle using topo sort
- B Longest path in DAG

MST

Priority Topic

- A Minimum spanning tree
- A Kruskal algorithm
- A Prim algorithm
- A DSU for Kruskal
- B Second best MST
- C Dynamic MST

DSU

Priority Topic

- A Union-find
- A Path compression
- A Union by rank/size
- A Connected components
- A Cycle detection
- B DSU on grid
- B DSU with rollback
- C Persistent DSU

Advanced Graphs

Priority Topic

- B Strongly connected components
- B Kosaraju algorithm
- B Tarjan SCC
- B Bridges
- B Articulation points

Priority Topic

- B Euler path/circuit
- B Bipartite matching
- C Max flow
- C Dinic algorithm
- C Min-cost max-flow
- C Hopcroft-Karp
- C 2-SAT

For placements, graph must-study:

BFS

DFS

Cycle detection

Bipartite graph

Topological sort

Dijkstra

DSU

Kruskal

Prim

Matrix BFS

For CP/advanced companies:

SCC

Bridges

Articulation points

0-1 BFS

Floyd-Warshall

Bellman-Ford

Max flow

17. Dynamic Programming

This is the biggest interview topic after arrays/graphs.

DP Basics

Priority Topic

- A Recursion to memoization
- A Tabulation
- A State definition
- A Transition
- A Base cases
- A Space optimization
- A 1D DP
- A 2D DP

Common DP Patterns

Priority Topic

- A Fibonacci style
- A Climbing stairs
- A House robber
- A Min/max path sum
- A Grid DP
- A Unique paths
- A Knapsack 0/1
- A Unbounded knapsack
- A Subset sum

Priority Topic

- A Partition equal subset sum
- A Coin change
- A Longest increasing subsequence
- A Longest common subsequence
- A Edit distance
- A Palindrome DP
- B Matrix chain multiplication
- B DP on trees
- B DP on graphs/DAG
- B Digit DP
- B Bitmask DP
- B Interval DP
- B Count DP
- B Probability DP
- C SOS DP
- C Convex hull trick optimization
- C Divide and conquer DP optimization
- C Knuth optimization
- C Profile DP
- C Rerooting DP

Must-practice DP patterns:

1D DP

Grid DP

Knapsack

LIS

LCS

Coin Change

Edit Distance

Palindrome Partitioning

Matrix Chain Multiplication

Tree DP

Bitmask DP

Important DP problems:

Climbing Stairs

House Robber

Coin Change

Longest Increasing Subsequence

Longest Common Subsequence

Edit Distance

0/1 Knapsack

Subset Sum

Partition Equal Subset Sum

Unique Paths

Minimum Path Sum

Palindromic Substrings

Burst Balloons

Matrix Chain Multiplication

18. Greedy Algorithms

Priority Topic

A Sorting-based greedy

Priority Topic

A	Interval scheduling
A	Activity selection
A	Minimum platforms
A	Meeting rooms
A	Fractional knapsack
A	Jump game
A	Gas station
A	Assign cookies
A	Merge intervals
A	Non-overlapping intervals
B	Huffman coding
B	Greedy with heap
B	Greedy with stack
B	Greedy on graphs
B	Greedy proof technique
C	Matroid greedy

Must-practice:

Activity selection

Meeting rooms

Jump game

Gas station

Minimum number of arrows

Task scheduler

Huffman coding

Greedy rule:

Greedy is not just choosing locally best.

You must know why the local choice is safe.

19. Divide and Conquer

Priority Topic

- A Merge sort
- A Quick sort
- A Binary search
- A Count inversions
- A Maximum subarray using divide and conquer
- B Closest pair of points
- B Karatsuba multiplication
- B Fast exponentiation
- C FFT
- C CDQ divide and conquer

Must-practice:

Merge sort

Quick sort

Count inversions

Median of two sorted arrays

Kth element in two sorted arrays

20. Tries

Priority Topic

- A Trie insert/search
- A Prefix search

Priority Topic

- A Word dictionary
- A Autocomplete
- B Delete from trie
- B XOR trie
- B Maximum XOR pair
- B Word break using trie
- C Persistent trie
- C Aho-Corasick

Must-practice:

Implement Trie

Word Search II

Replace Words

Maximum XOR of two numbers

Autocomplete system

21. Segment Tree

Priority Topic

- B Build segment tree
- B Range query
- B Point update
- B Range minimum query
- B Range sum query
- B Lazy propagation
- B Range update

Priority Topic

- C Segment tree with custom merge
- C Segment tree beats
- C Persistent segment tree
- C Dynamic segment tree
- C 2D segment tree

For placements, basic segment tree is usually enough.

Must know:

Range sum query

Range minimum query

Point update

Lazy propagation basics

22. Fenwick Tree / Binary Indexed Tree

Priority Topic

- B Prefix sum query
- B Point update
- B Range sum query
- B Count inversions
- B Coordinate compression with BIT
- C Range update range query
- C 2D BIT

Must-practice:

Count inversions

Number of smaller elements after self

Range sum mutable

23. Sparse Table

Priority Topic

- B Static range minimum query
- B Static range maximum query
- B GCD range query
- B Idempotent operations
- C LCA with sparse table

Use when:

Array does not change.

Many range min/max/gcd queries.

24. Disjoint Set Union

Already mentioned in graphs, but it is important enough separately.

Priority Topic

- A Parent array
- A Find
- A Union
- A Path compression
- A Union by size/rank
- A Cycle detection
- A Number of components
- B DSU on grid
- B Kruskal MST
- B Accounts merge

Priority Topic

B Redundant connection

C DSU rollback

C DSU on tree

Must-practice:

Number of provinces

Redundant connection

Accounts merge

Kruskal MST

Number of islands II

25. Advanced Data Structures

These are useful for CP or top-level interviews.

Priority Data Structure

B Ordered set

B Policy-based data structure

B LRU cache

B LFU cache

B Skip list idea

C Treap

C Splay tree

C Rope

C Persistent segment tree

C Heavy-Light Decomposition

C Centroid decomposition

Priority Data Structure

C Link-cut tree

C Wavelet tree

For placements, the important ones are:

LRU cache

LFU cache

Ordered set idea

26. Sliding Window

This is part of arrays/strings, but very important.

Priority Topic

A Fixed-size window

A Variable-size window

A Longest substring without repeating

A Minimum window substring

A Count subarrays with condition

A At most K distinct

B Exactly K = atMost(K) - atMost(K-1)

B Monotonic deque window

B Sliding window maximum

Must-practice:

Longest substring without repeating characters

Minimum window substring

Longest repeating character replacement

Max consecutive ones

Subarrays with K distinct

Sliding window maximum

27. Two Pointers

Priority Topic

- A Opposite direction pointers
- A Same direction pointers
- A Fast and slow pointers
- A Sorted array pair sum
- A Remove duplicates
- A Container with most water
- A Trapping rain water
- A Dutch national flag
- B 3Sum
- B 4Sum
- B Linked list cycle
- B Palindrome checking

Must-practice:

Two sum sorted

3Sum

Container with most water

Trapping rain water

Remove duplicates

Sort colors

Linked list cycle

28. Intervals and Sweep Line

Priority Topic

- A Merge intervals
- A Insert interval
- A Non-overlapping intervals
- A Meeting rooms
- A Minimum platforms
- B Sweep line
- B Difference array events
- B Line sweep with sorting
- B Calendar booking
- C Segment tree with sweep line

Must-practice:

Merge intervals

Meeting rooms II

Minimum platforms

Insert interval

Car pooling

My calendar

29. Matrix / Grid Algorithms

Priority Topic

- A Matrix traversal
- A Spiral traversal
- A Rotate matrix
- A Search in sorted matrix

Priority Topic

- A Flood fill
- A Number of islands
- A BFS on grid
- A DFS on grid
- A Shortest path in grid
- B Multi-source BFS
- B DP on grid
- B Prefix sum 2D
- B Connected components in grid
- C Bitmask grid DP
- C Profile DP

Must-practice:

Number of islands

Rotten oranges

Flood fill

Word search

Shortest path in binary matrix

Minimum path sum

Unique paths

30. Searching Algorithms

Priority Topic

- A Linear search
- A Binary search

Priority Topic

- A Search in rotated array
- A Search in 2D matrix
- A Lower/upper bound
- B Ternary search
- B Exponential search
- B Jump search
- C Interpolation search

Main thing:

Binary search on answer is more important than normal binary search.

31. Randomized Algorithms

Priority Topic

- B Randomized quicksort
- B Reservoir sampling
- B Shuffle algorithm
- B Random pick with weight
- C Treap
- C Monte Carlo idea
- C Las Vegas algorithm idea

For placements, know:

Reservoir sampling

Fisher-Yates shuffle

Randomized quicksort

32. Geometry

Mostly CP, rarely placement.

Priority Topic

- C Points and vectors
- C Dot product
- C Cross product
- C Orientation test
- C Line intersection
- C Convex hull
- C Polygon area
- C Closest pair of points
- C Sweep line geometry

For placements, geometry is low priority.

33. Game Theory

Mostly CP.

Priority Topic

- B Basic winning/losing states
- B Nim game
- B XOR strategy
- C Grundy numbers
- C Sprague-Grundy theorem
- C Minimax
- C Alpha-beta pruning

For placements, just know:

Nim game

Minimax basics

34. Probability and Random Processes

Rare in normal SDE placements, useful for quant/HFT/ML roles.

Priority Topic

- B Basic probability
- B Expected value
- B Linearity of expectation
- B Random variables
- B Conditional probability
- B Bayes theorem
- C Markov chains
- C Coupon collector
- C Random walks

For interviews:

Expected value + linearity of expectation is most important.

35. Common Interview Design Data Structures

These are implementation-heavy.

Priority Problem

- A Min stack
- A Queue using stacks
- A Stack using queues
- A LRU cache
- A LFU cache
- A Trie

Priority Problem

- A Add and search word
- A RandomizedSet
- A MedianFinder
- B AllOne data structure
- B Time-based key-value store
- B Snapshot array
- B Design Twitter
- B Design browser history
- B Design file system
- B Design autocomplete system

These are very important for placements because they test DS + clean coding.

36. Must-Know Patterns

This is the most important section. Most interview problems are pattern-based.

Priority Pattern

- A Prefix sum
- A Hash map counting
- A Two pointers
- A Sliding window
- A Binary search
- A Binary search on answer
- A Sorting + greedy
- A Monotonic stack
- A Heap / top K

Priority Pattern

A	BFS
A	DFS
A	Backtracking
A	DP
A	Tree recursion
A	Graph traversal
A	Union-find
B	Trie
B	Interval merge
B	Sweep line
B	Topological sort
B	Shortest path
B	Bitmask
B	Tree DP
B	LCA
B	Segment tree
B	Fenwick tree
C	String matching
C	Digit DP
C	Flow
C	Advanced DP optimization

37. Priority Study Order

Study in this exact order.

Stage 1 — Absolute must

1. Complexity analysis
2. Arrays
3. Strings
4. Hashing
5. Two pointers
6. Sliding window
7. Sorting
8. Binary search
9. Stack
10. Queue/deque

Stage 2 — Core interview DSA

11. Linked list
12. Recursion
13. Backtracking
14. Trees
15. BST
16. Heap
17. Greedy
18. Dynamic programming basics

Stage 3 — Graph + advanced interview

19. Graph BFS/DFS
20. Matrix BFS/DFS
21. Topological sort
22. Dijkstra
23. DSU
24. MST
25. Advanced DP patterns

Stage 4 — Strong company / CP-heavy

- 26. Trie
- 27. Segment tree
- 28. Fenwick tree
- 29. Sparse table
- 30. String algorithms
- 31. Bitmask DP
- 32. LCA
- 33. SCC/bridges/articulation points
- 34. 0-1 BFS
- 35. Flow/matching

Stage 5 — Bonus

- 36. Geometry
- 37. Game theory
- 38. Probability
- 39. Randomized algorithms
- 40. Advanced number theory
- 41. Advanced DP optimization

38. Minimum Problem Count

For good placement readiness:

Area	Problems
Arrays + strings	50
Hashing	20
Binary search	25
Sliding window/two pointers	30
Stack/queue	25

Area	Problems
Linked list	20
Trees/BST	40
Heap	20
Graphs	45
DP	60
Greedy	30
Backtracking	20
Bit manipulation	15
Math	20
Trie/segment tree/BIT	20
Design DS problems	15

Total:

Around 430–450 problems for very strong preparation.

But if time is limited, do this:

200 high-quality problems > 500 random problems

39. Minimum Set for Placements

If you want the **minimum complete placement set**, study these:

Arrays

Strings

Hashing

Two pointers

Sliding window

Binary search

Sorting

Stack

Queue

Linked list

Trees

BST

Heap

Graphs

Dynamic programming

Greedy

Backtracking

Bit manipulation

Math basics

Trie basics

DSU

Topological sort

Dijkstra

SQL-style logical thinking

This is enough for most SDE placements.

40. Advanced Set for Top Companies

For Google/Microsoft/Atlassian/Amazon-level strong prep, add:

Binary search on answer

Monotonic stack

Monotonic deque

Advanced graph BFS

Multi-source BFS

0-1 BFS

Tree DP

Bitmask DP

Interval DP

LCA

Segment tree

Fenwick tree

String hashing

KMP/Z algorithm

Trie + XOR trie

System design DS problems

For CP/HFT/quant-style prep, add:

SCC

Bridges/articulation points

Max flow

Bipartite matching

Number theory

Probability

Expected value

Matrix exponentiation

Game theory

Geometry

Advanced DP optimization

41. Your Best Personal Focus

Since you already have strong CP background, your weak point is probably not algorithms. For placements, focus more on:

1. Explaining the approach clearly
2. Writing clean code
3. Handling edge cases

4. Solving under 30–40 minutes
5. Practicing LeetCode-style interviews
6. DP and graph pattern recognition
7. Design data structures like LRU, Trie, MedianFinder

Your highest ROI list:

Arrays + Hashing

Binary Search on Answer

Sliding Window

Trees

Graphs

DP

Heap

Monotonic Stack

Trie

DSU

Design DS Problems